

Розділ 12

Мережеві програми

У цій книзі багато прикладів стосуються зчитування файлів і пошуку даних у них, однак в інтернеті також можна знайти багато різних джерел інформації.

У цьому розділі спробуємо зобразити роботу браузера та завантажити вебсторінки за допомогою Hypertext Transfer Protocol (HTTP) – протоколу передачі гіпертексту. Потім зчитуємо дані вебсторінки та виконаємо парсинг.

12.1 HTTP – Протокол передачі гіпертексту

Мережевий протокол, який забезпечує роботу інтернету, насправді досить простий. У Python є вбудована підтримка сокетів (модуль `socket`), що дозволяє дуже легко встановлювати мережеві з'єднання та отримувати дані через ці сокети у програмі на Python.

Робота із *сокетами* дуже схожа на роботу з файлами, за винятком того, що один сокет забезпечує двостороннє з'єднання між двома програмами. Можна як зчитувати, так і писати в один і той самий сокет. Дані, записані в сокет, надсилаються до застосунку на іншому кінці сокета. При зчитуванні з сокета ви отримуєте дані, які надіслав інший застосунок.

Однак, якщо спробувати зчитати сокет¹, коли програма на іншому кінці сокета не надіслала дані, доведеться просто чекати. Якщо програми на обох кінцях сокета просто вичікуватимуть на якісь дані, нічого не надсилаючи, то чекатимуть дуже довго, тож важливою частиною програм, які взаємодіють через інтернет, є наявність певного протоколу.

Протокол – це набір чітких правил, які визначають, хто має надсилати повідомлення першим, що слід робити, якою має бути відповідь на це повідомлення, хто надсилає наступним тощо. Можна сказати, два застосунки на обох кінцях сокета танцюють і намагаються не наступати одне одному на пальці.

Існує безліч документів, які описують ці мережеві протоколи. Протокол передачі гіпертексту описано в наступному документі:

<https://www.w3.org/Protocols/rfc2616/rfc2616.txt>

¹ Якщо бажаєте дізнатися більше про сокети, протоколи чи розробку вебсерверів, пройдіть курс за посиланням <https://www.dj4e.com>.

Це довгий і складний документ обсягом 176 сторінок з великою кількістю деталей. Якщо вам цікаво, можете прочитати повністю. Проте, якщо зазирнути на сторінку 36 RFC2616, можна знайти синтаксис запиту GET. Щоб зробити запит на документ із вебсервера, ми встановлюємо з'єднання, наприклад, з сервером www.pr4e.org на порт 80, а потім надсилаємо рядок такого вигляду

```
GET http://data.pr4e.org/romeo.txt HTTP/1.0
```

де другим параметром є вебсторінка, на яку робимо запит, а далі також надсилаємо порожній рядок. Вебсервер відповість певною інформацією про документ у заголовку та порожнім рядком, за яким буде вміст документа.

12.2 Найпростіший у світі веббраузер

Найбільш простий спосіб показати, як працює протокол HTTP – написати нескладну програму Python, яка встановлює з'єднання з вебсервером і за правилами протоколу HTTP запитує документ та відображає те, що сервер повертає назад.

```
import socket

mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
mysock.connect(('data.pr4e.org', 80))
cmd = 'GET http://data.pr4e.org/romeo.txt HTTP/1.0\r\n\r\n'.encode()
mysock.send(cmd)

while True:
    data = mysock.recv(512)
    if len(data) < 1:
        break
    print(data.decode(), end=' ')

mysock.close()
```

Код: <https://www.py4e.com/code3/socket1.py>

Спочатку програма встановлює з'єднання з портом 80 на сервері www.pr4e.com. Оскільки наша програма грає роль веббраузера, згідно з протоколом HTTP, необхідно надіслати команду **GET**, за якою повинен бути порожній рядок. `\r\n` означає EOL (end of line - кінець рядка), тому `\r\n\r\n` вказує на те, що між двома послідовностями EOL нічого немає. Це рівнозначно порожньому рядку.

Після надсилання порожнього рядка пишемо цикл, який отримує дані із сокета фрагментами по 512 символів і виводить їх на екран, доки не закінчатся дані для зчитування (тобто `recv()` повертає порожній рядок).

Програма виводить такий результат:

12.2. НАЙПРОСТІШИЙ У СВІТІ ВЕББРАУЗЕР

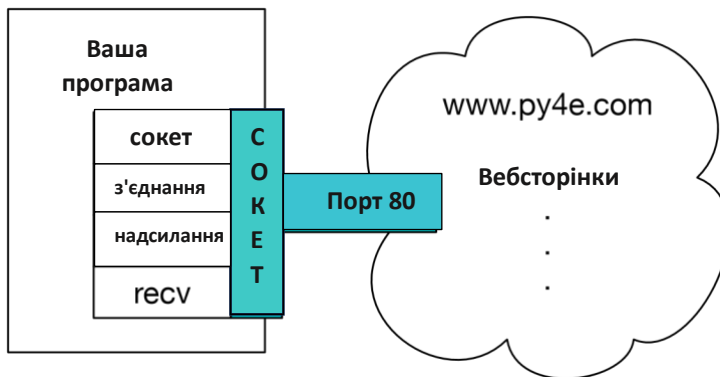


Рисунок 12.1. З'єднання сокета

```
HTTP/1.1 200 OK
Date: Wed, 11 Apr 2018 18:52:55 GMT
Server: Apache/2.4.7 (Ubuntu)
Last-Modified: Sat, 13 May 2017 11:22:22 GMT
ETag: "a7-54f6609245537"
Accept-Ranges: bytes
Content-Length: 167
Cache-Control: max-age=0, no-cache, no-store, must-revalidate
Pragma: no-cache
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Connection: close
Content-Type: text/plain
```

```
But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief
```

Вихідні дані починаються із заголовків, які вебсервер надсилає для опису документа. Наприклад, заголовок **Content-Type** вказує на те, що документ є звичайним текстовим документом (**text/plain**).

Після того, як сервер надсилає нам заголовки, він додає порожній рядок, щоб позначити кінець заголовків, а потім надсилає власне дані файлу *romeo.txt*.

У цьому прикладі показано, як створити низькорівневе мережеве з'єднання за допомогою сокетів. Сокети можна використовувати для зв'язку з вебсервером, поштовим сервером чи іншими типами серверів. Все, що потрібно – це знайти документ, який описує протокол, і, відповідно до нього, написати код для надсилання та отримання даних.

Утім, оскільки найчастіше використовується вебпротокол HTTP, у Python є спеціальна бібліотека, розроблена безпосередньо для його підтримки для отримання документів і даних через інтернет.

Одна з вимог використання протоколу HTTP – необхідність надсилати та отримувати дані у вигляді байтових об'єктів, а не рядків. У попередньому прикладі методи **encode()** та **decode()** перетворюють рядки в байт-об'єкти і навпаки.

У наступному прикладі використовується позначення `b' '`, щоб вказати, що змінну слід зберігати як байтовий об'єкт. `encode()` та `b' '` еквівалентні.

```
>>> b'Hello world'
b'Hello world'
>>> 'Hello world'.encode()
b'Hello world'
```

12.3 Отримання зображення через HTTP

У наведеному вище прикладі ми отримували звичайний текстовий файл, який містив нові рядки, і просто скопіювали дані на екран під час запуску програми. Подібну програму можна використати для отримання зображення через HTTP. Замість того, щоб копіювати дані на екран під час роботи програми, слід зібрати їх у рядок, відкинути заголовки, а потім зберегти дані зображення у файл, як показано нижче:

```
import socket
import time

HOST = 'data.pr4e.org'
PORT = 80
mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
mysock.connect((HOST, PORT))
mysock.sendall(b'GET http://data.pr4e.org/cover3.jpg HTTP/1.0\r\n\r\n')
count = 0
picture = b""

while True:
    data = mysock.recv(5120)
    if len(data) < 1: break
    #time.sleep(0.25)
    count = count + len(data)
    print(len(data), count)
    picture = picture + data

mysock.close()

# Пошук кінця заголовка (2 CRLF)
pos = picture.find(b"\r\n\r\n")
print('Довжина заголовка', pos)
print(picture[:pos].decode())

# Пропускаємо заголовок і зберігаємо дані зображення
picture = picture[pos+4:]
fhand = open("stuff.jpg", "wb")
fhand.write(picture)
fhand.close()

# Код: https://www.py4e.com/code3/urljpeg.py
```

Після запуску, програма видає такий результат:

12.3. ОТРИМАННЯ ЗОБРАЖЕННЯ ЧЕРЕЗ HTTP

```
$ python urljpeg.py
5120 5120
5120 10240
4240 14480
5120 19600
...
5120 214000
3200 217200
5120 222320
5120 227440
3167 230607
Header length 393
HTTP/1.1 200 OK
Date: Wed, 11 Apr 2018 18:54:09 GMT
Server: Apache/2.4.7 (Ubuntu)
Last-Modified: Mon, 15 May 2017 12:27:40 GMT
ETag: "38342-54f8f2e5b6277"
Accept-Ranges: bytes
Content-Length: 230210
Vary: Accept-Encoding
Cache-Control: max-age=0, no-cache, no-store, must-revalidate
Pragma: no-cache
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Connection: close
Content-Type: image/jpeg
```

Бачимо, що для цієї адреси заголовок **"Content-Type"** вказує, що документ є зображенням (**image/jpeg**). Після завершення роботи програми можна переглянути дані зображення, відкривши файл **stuff.jpg** у програмі для перегляду зображень.

Під час роботи програми можете побачити, що ми не отримуємо 5120 символів щоразу, коли викликаємо метод **recv()**. Ми отримуємо стільки символів, скільки було передано нам мережею вебсервером у момент виклику **recv()**. У цьому прикладі ми отримуємо лише 3200 символів щоразу, коли робимо запит на 5120 символів даних.

Ваші результати можуть відрізнятися залежно від швидкості мережі. Також, зверніть увагу, що під час останнього виклику **recv()** ми отримуємо 3167 байтів, тобто кінець потоку, а під час наступного виклику **recv()** – рядок з нульовою довжиною, що означає, що сервер викликав **close()** на своєму кінці сокета та більше не приймає дані.

Можна сповільнити наші послідовні виклики **recv()**, якщо не закоментувати виклик **time.sleep()**. Таким чином, після кожного виклику чекаємо чверть секунди, щоб сервер міг «випередити» нас і надіслати більше даних, перш ніж ми знову виконаємо виклик **recv()**. З такою затримкою програма буде працювати так:

```
$ python urljpeg.py
5120 5120
5120 10240
5120 15360
...
```

```

5120 225280
5120 230400
207 230607
Header length 393
HTTP/1.1 200 OK
Date: Wed, 11 Apr 2018 21:42:08 GMT
Server: Apache/2.4.7 (Ubuntu)
Last-Modified: Mon, 15 May 2017 12:27:40 GMT
ETag: "38342-54f8f2e5b6277"
Accept-Ranges: bytes
Content-Length: 230210
Vary: Accept-Encoding
Cache-Control: max-age=0, no-cache, no-store, must-revalidate
Pragma: no-cache
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Connection: close
Content-Type: image/jpeg

```

Тепер, окрім першого та останнього викликів `recv()`, ми отримуємо 5120 символів щоразу, як запитуємо нові дані.

Між сервером, який робить запити `send()`, і нашим застосунком, який робить запити `recv()`, є буфер. Під час запуску програми із встановленою затримкою, в певний момент сервер може заповнити буфер у сокеті і вимушено призупинити роботу, доки наша програма не почне очищати буфер. Призупинення роботи програми-відправника або програми-отримувача називається «керування потоком».

12.4 Отримання вебсторінок за допомогою `urllib`

Попри те, що можна вручну надсилати та отримувати дані через HTTP за допомогою бібліотеки сокетів, існує набагато простіший спосіб виконати це в Python за допомогою бібліотеки `urllib`.

Завдяки `urllib` можна працювати з вебсторінкою так само як з файлом. Потрібно лише вказати вебсторінку, яку бажаєте отримати, а `urllib` опрацює увесь HTTP-протокол та деталі заголовків.

Нижче показано еквівалентний код для зчитування файлу `romeo.txt` з інтернету за допомогою `urllib`:

```

import urllib.request

fhand = urllib.request.urlopen('http://data.pr4e.org/romeo.txt')
for line in fhand:
    print(line.decode().strip())

# Код: https://www.py4e.com/code3/urllib1.py

```

Після того, як вебсторінку було відкрито за допомогою `urllib.request.urlopen`, можемо обробляти її як файл і зчитувати його за допомогою циклу `for`.

12.5. ЗЧИТУВАННЯ ДВІЙКОВИХ ФАЙЛІВ ЗА ДОПОМОГОЮ URLLIB

Під час роботи програми видно лише виведений вміст файлу. Заголовки все ще надсилаються, але код `urllib` поглинає їх і повертає нам лише дані.

```
But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief
```

Наприклад, можна написати програму для отримання даних для файлу `romeo.txt` і обчислити частоту кожного слова у файлі таким чином:

```
import urllib.request, urllib.parse, urllib.error

fhand = urllib.request.urlopen('http://data.pr4e.org/romeo.txt')

counts = dict()
for line in fhand:
    words = line.decode().split()
    for word in words:
        counts[word] = counts.get(word, 0) + 1
print(counts)
```

Код: <https://www.py4e.com/code3/urlwords.py>

Знов-таки, відкривши вебсторінку, її можна зчитувати як локальний файл.

12.5 Зчитування двійкових файлів за допомогою urllib

Іноді виникає потреба витягти нетекстовий (або бінарний) файл, наприклад, зображення чи відео. Дані з таких файлів переважно не призначені для виведення, але за допомогою програми `urllib` можна створити копію URL-адреси до локального файлу на жорсткому диску.

Принцип роботи такий: відкриваємо URL-адресу, за допомогою `read` завантажуюмо увесь вміст документа у змінну рядка (`img`), а потім записуємо цю інформацію до локального файлу, як показано нижче:

```
import urllib.request, urllib.parse, urllib.error

img = urllib.request.urlopen('http://data.pr4e.org/cover3.jpg').read()
fhand = open('cover3.jpg', 'wb')
fhand.write(img)
fhand.close()
```

Код: <https://www.py4e.com/code3/curl1.py>

Ця програма за раз зчитує всі дані через мережу та зберігає їх у змінній `img` в основній пам'яті комп'ютера, потім відкриває файл `cover3.jpg` і записує дані на диск. Аргумент `wb` для `open()` відкриває бінарний файл лише для запису. Ця програма працюватиме за умови, що розмір файлу менший за розмір пам'яті вашого комп'ютера.

Утім, якщо відео- чи аудіофайл надто великий, програма може вийти з ладу або, щонайменше, працювати дуже повільно, коли на комп'ютері закінчуватиметься пам'ять. Щоб пам'ять не

вичерпалася, витягуємо дані блоками (чи буферами) та записуємо кожен блок на диск перш ніж перейти до наступного блоку. Таким чином, програма може зчитувати файли будь-якого розміру, не використовуючи всю пам'ять комп'ютера.

```
import urllib.request, urllib.parse, urllib.error

img = urllib.request.urlopen('http://data.pr4e.org/cover3.jpg')
fhand = open('cover3.jpg', 'wb')
size = 0
while True:
    info = img.read(100000)
    if len(info) < 1: break
    size = size + len(info)
    fhand.write(info)

print(size, 'символів скопійовано.')
fhand.close()
```

Код: <https://www.py4e.com/code3/curl2.py>

У цьому прикладі за раз зчитується лише 100000 символів, потім вони записуються у файл **cover3.jpg** до того, як з'являться наступні 100000 символів даних з інтернету. Ця програма працює так:

```
python curl2.py
230210 characters copied.
```

12.6 Парсинг HTML і вебскрепінг

У Python, **urllib** часто використовується для *скрепінгу* вебсторінок. Вебскрепінг — це написання програми, яка імітує веббраузер і отримує сторінки, а потім обробляє дані цих сторінок у пошуках шаблонів.

Наприклад, пошукова система, як-от Google, бачить джерело однієї вебсторінки, витягує з нього посилання на інші сторінки, потім отримує ці сторінки, витягуючи посилання і так далі. За допомогою цього Google *плете* свій шлях, ніби павук, через майже всі сторінки в інтернеті.

Google також використовує частоту посилань зі знайдених сторінок на певну сторінку як один з показників того, наскільки «важливою» є ця сторінка і як високо вона повинна відображатися в результатах пошуку.

12.7 Парсинг HTML за допомогою регулярних виразів

Одним із найпростіших способів парсингу HTML є застосування регулярних виразів для багаторазового пошуку та витягування підрядків, які відповідають певному шаблону.

Ось проста вебсторінка:

12.7. ПАРСИНГ HTML ЗА ДОПОМОГОЮ РЕГУЛЯРНИХ ВИРАЗІВ

```
<h1> The First Page </h1>
<p>
If you like, you can switch to the
<a href="http://www.dr-chuck.com/page2.htm">
Second Page </a>.
</p>
```

Для наведеного вище тексту можна сформувати чіткий регулярний вираз для пошуку та витягування значень посилань:

```
href="http[s]?://.+?"
```

Цей регулярний вираз шукає рядки, які починаються з "href="http://" або "href="https://", після яких йде один або більше символів (.+?) та ще одні подвійні лапки. Знак питання після [s]? вказує на пошук рядка «http», за яким стоїть нуль або один символ «s».

Знак питання додається до .+?, щоб вказати, що пошук буде виконуватися у «нежадібний» спосіб. Під час нежадібного способу здійснюється пошук *найменшого* можливого рядка, а під час жадібного – *найбільшого*.

Додаємо круглі дужки до нашого регулярного виразу, щоб вказати, яку частину знайденого рядка потрібно витягти, і отримуємо таку програму:

```
# Здійснюємо пошук значень посилань у введених URL-адресах
import urllib.request, urllib.parse, urllib.error
import re
import ssl

# Ігноруємо помилки сертифіката SSL
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE

url = input('Введення - ')
html = urllib.request.urlopen(url, context=ctx).read()
links = re.findall(b'href="(http[s]?://.*?)"', html)
for link in links:
    print(link.decode())

# Код: https://www.py4e.com/code3/urlregex.py
```

Завдяки бібліотеці **ssl** ця програма отримує доступ до вебсайтів, які суворо дотримуються протоколу HTTPS. Метод **read** повертає вихідний код HTML як байтовий об'єкт замість об'єкта **HTTPResponse**. Метод **findall** надає список усіх рядків, які відповідають нашому регулярному виразу, повертаючи лише текст посилання, що був у подвійних лапках.

Після запуску програми та введення URL-адреси маємо такий результат:

Введення – <https://docs.python.org>
<https://docs.python.org/3/index.html>
<https://www.python.org/>
<https://docs.python.org/3.8/>
<https://docs.python.org/3.7/>
<https://docs.python.org/3.5/>
<https://docs.python.org/2.7/>
<https://www.python.org/doc/versions/>
<https://www.python.org/dev/peps/>
<https://wiki.python.org/moin/BeginnersGuide>
<https://wiki.python.org/moin/PythonBooks>
<https://www.python.org/doc/av/>
<https://www.python.org/>
<https://www.python.org/psf/donations/>
<http://sphinx.pocoo.org/>

Застосування регулярних виразів чудово працює, коли HTML правильно відформатований і зрозумілий. Однак, зважаючи на велику кількість «пошкоджених» HTML-сторінок, у разі використання лише регулярних виразів можна пропустити деякі валідні посилання чи отримати неякісні дані.

Цю проблему можна вирішити за допомогою надійної бібліотеки для парсингу HTML.

12.8 Парсинг HTML за допомогою BeautifulSoup

Попри те, що HTML виглядає як XML², а деякі сторінки навіть розроблені як XML, більшість HTML-сторінок мають різноманітні вади, що спричиняє відхилення парсером XML всієї сторінки HTML через неправильну побудову.

Існує ряд бібліотек Python, які здатні допомогти у парсингу HTML та витягування даних зі сторінок. Кожна з бібліотек має свої переваги та недоліки, можете обрати одну з них відповідно до ваших потреб.

Для прикладу здійснимо парсинг вхідних даних HTML і витягнемо посилання за допомогою бібліотеки *BeautifulSoup*. Вона допускає наявність значних помилок у HTML і водночас дозволяє легко витягувати потрібні дані. Завантажити та встановити код BeautifulSoup можна за посиланням:

<https://pypi.python.org/pypi/beautifulsoup4>

Інформація про встановлення BeautifulSoup за допомогою Python Package Index tool **pip** доступна на сайті:

<https://packaging.python.org/tutorials/installing-packages/>

Для зчитування сторінки скористаємося **urllib**, а для витягування атрибутів **href** з якірних тегів (a) – **BeautifulSoup**.

Для запуску завантажте zip-файл BeautifulSoup
<http://www.py4e.com/code3/bs4.zip>
і розархівуйте його у той самий каталог, що й цей файл

² Формат XML описано в наступному розділі.

12.8. ПАРСИНГ HTML ЗА ДОПОМОГОЮ BEAUTIFULSOUP

```

import urllib.request, urllib.parse, urllib.error
from bs4 import BeautifulSoup
import ssl

# Ігноруємо помилки сертифіката SSL
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE

url = input('Введення - ')
html = urllib.request.urlopen(url, context=ctx).read()
soup = BeautifulSoup(html, 'html.parser')

# Вибудуємо усі якірні теги
tags = soup('a')
for tag in tags:
    print(tag.get('href', None))

# Код: https://www.py4e.com/code3/urllinks.py

```

Програма робить запит на вебадресу, потім відкриває вебсторінку, зчитує дані і передає їх парсеру BeautifulSoup, а далі знаходить усі якірні теги і виводить атрибут **href** для кожного з них.

Програма виводить на екран наступне:

```

Введення - https://docs.python.org
genindex.html
py-modindex.html
https://www.python.org/
#
whatsnew/3.6.html
whatsnew/index.html
tutorial/index.html
library/index.html
reference/index.html
using/index.html
howto/index.html
installing/index.html
distributing/index.html
extending/index.html
c-api/index.html
faq/index.html
py-modindex.html
genindex.html
glossary.html
search.html
contents.html
bugs.html
about.html
license.html
copyright.html

```

```

download.html
https://docs.python.org/3.8/
https://docs.python.org/3.7/
https://docs.python.org/3.5/
https://docs.python.org/2.7/
https://www.python.org/doc/versions/
https://www.python.org/dev/peps/
https://wiki.python.org/moin/BeginnersGuide
https://wiki.python.org/moin/PythonBooks
https://www.python.org/doc/av/
genindex.html
py-modindex.html
https://www.python.org/
#
copyright.html
https://www.python.org/psf/donations/
bugs.html
http://sphinx.pocoo.org/

```

Цей список набагато довший, адже деякі якірні теги HTML є відносними шляхами (наприклад, `tutorial/index.html`) або внутрішніми посиланнями (наприклад, `#`), які не містять «`http://`» чи «`https://`», що було вимогою в нашому регулярному виразі.

Крім того, BeautifulSoup можна використовувати, щоб витягнути різні частини кожного теґу:

```

# Для запуску завантажте zip-файл BeautifulSoup
# http://www.py4e.com/code3/bs4.zip
# і розархівуйте його у той самий каталог, що й цей файл

```

```

from urllib.request import urlopen
from bs4 import BeautifulSoup
import ssl

# Ігноруємо помилки сертифіката SSL
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE
url = input('Введення - ')
html = urlopen(url, context=ctx).read()
soup = BeautifulSoup(html, "html.parser")

```

```

# Видобуваємо усі якірні теґи

```

```

tags = soup('a')
for tag in tags:

```

```

# Виокремлюємо різні частини теґів

```

```

print('TAG:', tag)
print('URL:', tag.get('href', None))
print('Бміст:', tag.contents[0])
print('Attrs:', tag.attrs)

```

```

# Код: https://www.py4e.com/code3/urllink2.py

```

```

python urllink2.py

```

```
Введення - http://www.dr-chuck.com/page1.htm
TAG: <a href="http://www.dr-chuck.com/page2.htm">
Друга сторінка</a>
URL: http://www.dr-chuck.com/page2.htm
Вміст: ['\nДруга сторінка']
Attrs: [('href', 'http://www.dr-chuck.com/page2.htm')]
```

html.parser – парсер HTML, що входить до стандартної бібліотеки Python 3. Інформацію про інші парсери HTML можна знайти за посиланням:

<http://www.crummy.com/software/BeautifulSoup/bs4/doc/#installing-a-parser>

Ці приклади лише частково демонструють потужність BeautifulSoup, коли йдеться про парсинг HTML.

12.9 Корисне для користувачів Unix/Linux

Якщо ви користуєтеся комп'ютером з Linux, Unix або Macintosh, у вашій операційній системі, ймовірно, є вбудовані команди, які витягують як звичайний текст, так і бінарні файли за допомогою протоколів HTTP або File Transfer (FTP). Однією з таких команд є **curl**:

```
$ curl -O http://www.py4e.com/cover.jpg
```

Команда **curl** – це скорочення від «сору URL» (укр. – скопіювати URL), тому два наведені раніше приклади для витягання бінарних файлів за допомогою **urllib** на сайті www.py4e.com/code3 названо **curl1.py** та **curl2.py**, бо вони виконують функції, подібні команді **curl**. Існує також зразок програми **curl3.py**, яка виконує цю дію трохи ефективніше, у разі, якщо потрібно використати цей шаблон у програмі, яку ви пишете.

Друга подібна команда – **wget**:

```
$ wget http://www.py4e.com/cover.jpg
```

Ці команди спрощують витягування вебсторінок і віддалених файлів.

12.10 Словник

BeautifulSoup Бібліотека Python для парсингу HTML-документів і витягування даних з HTML-документів, що виправляє більшість недосконалостей HTML, які браузері зазвичай ігнорують. Завантажити код BeautifulSoup можна за посиланням www.crummy.com.

port (порт) Число, яке вказує на те, з якою програмою ви зв'язуєтеся, коли встановлюєте з'єднання з сервером через сокет. Наприклад, вебтрафік зазвичай використовує порт 80, а поштовий трафік – порт 25.

scrape (скрепінг) Процес, під час якого програма відтворює веббраузер і завантажує вебсторінку, а потім переглядає її вміст. Часто програми переходять за посиланнями на одній сторінці, щоб знайти наступну сторінку, таким чином вони можуть переходити через мережу сторінок або соціальну мережу.

socket (сокет) Мережеве з'єднання між двома застосунками, де вони можуть надсилати та отримувати дані в обох напрямках.

spider (плетіння павутини) Дія пошукової системи, що знаходить сторінку, а потім усі інші сторінки, на які є посилання на цій сторінці, і так доти, доки не отримає майже всі сторінки в інтернеті, які вона використовує для побудови свого пошукового індексу.

12.11 Вправи

Вправа 1: Змініть програму сокета `socket1.py` так, щоб вона запитувала у користувачів URL-адресу, аби вони могли зчитати будь-яку вебсторінку. Можете використовувати `split('/')` для поділу URL-адреси на частини, щоб отримати ім'я хоста для виклику з'єднання з сокетом. Додайте перевірку помилок за допомогою `try` та `except` для обробки умов, коли користувач вводить неправильно відформатовану або вигадану URL-адресу.

Вправа 2: Змініть вашу програму для роботи з сокетами так, щоб вона підраховувала кількість отриманих символів і припиняла виводити будь-який текст після того, як буде показано 3000 символів. Програма повинна витягати весь документ, підраховувати загальну кількість символів і виводити їх кількість в кінці документа.

Вправа 3: Скористайтеся `urllib`, щоб повторити попередню вправу: слід (1) витягнути документ з URL-адреси, (2) відобразити до 3000 символів і (3) підрахувати загальну кількість символів у документі. У цій вправі не звертайте уваги на заголовки, просто відобразіть перші 3000 символів вмісту документа.

Вправа 4: Змініть програму `urllinks.py` так, щоб вона витягувала і підраховувала теги абзаців (p) з отриманого HTML-документа і виводила кількість абзаців як результат роботи програми. Не виводьте сам текст абзаців, а лише підрахуйте їхню кількість. Протестуйте програму на кількох невеликих та великих вебсторінках.

Вправа 5: (Підвищена складність) Змініть програму роботи з сокетами так, щоб вона показувала дані лише після прийому заголовків і порожнього рядка. Не забувайте, `recv` приймає символи (символи переходу на новий рядок тощо), а не рядки.